

КУРСОВОЙ ПРОЕКТ

по дисциплине “Параллельные вычислительные технологии”

на тему

Разработка параллельной MPI-программы решения СЛАУ методом Якоби

Выполнил

студент

Полякова Кристина

Новосибирск – 2025

СОДЕРЖАНИЕ

Введение	4
1 ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ И MPI	5
2 МЕТОД ЯКОБИ РЕШЕНИЯ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ	6
2.1 Принципы математической реализации метода	6
2.2 Алгоритмическая сложность	6
2.3 Условия сходимости	7
3 ПАРАЛЛЕЛЬНАЯ РЕАЛИЗАЦИЯ МЕТОДА ЯКОБИ	8
3.1 Декомпозиция по данным	8
3.2 Разделение матрицы между процессами	8
3.3 Межпроцессорное взаимодействие	8
4 РАЗРАБОТКА MPI-ПРОГРАММЫ	10
5 ЭКСПЕРИМЕНТАЛЬНЫЕ ИССЛЕДОВАНИЯ	11
5.1 Вычислительная платформа (кластер Pine)	11
5.2 Методика проведения экспериментов	11
5.3 Расчёт ускорения	12
5.4 Влияние размера задачи на масштабируемость	13
ВЫВОД	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	15

6	ПРИЛОЖЕНИЯ	16
---	----------------------	----

Введение

Современные вычислительные задачи характеризуются большим объёмом данных и высокой вычислительной сложностью, что требует применения параллельных методов и архитектур. Одной из ключевых задач линейной алгебры является решение систем линейных алгебраических уравнений (СЛАУ), которые возникают при моделировании физических процессов, оптимизации и обработке данных.

Для эффективного решения СЛАУ большого размера применяются параллельные алгоритмы, реализованные с использованием стандарта MPI (Message Passing Interface), который обеспечивает распределённое выполнение программы на кластере или многопроцессорной системе.

Целью данной работы является разработка параллельной MPI-программы решения СЛАУ методом Якоби, а также проведение экспериментов для оценки эффективности распараллеливания.

Для достижения цели в работе решены следующие задачи:

- изучены основы параллельных вычислений и обменов MPI;
- рассмотрены особенности метода Якоби;
- разработан параллельный алгоритм решения СЛАУ;
- реализована MPI-программа на языке C;
- проведено моделирование работы алгоритма на разных числах процессов;
- выполнен анализ ускорения и вычислительной эффективности.

1 ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ И MPI

MPI — стандарт, определяющий функции для обмена информацией между процессами в системах с распределённой памятью. Его ключевые возможности:

1. запуск множества процессов (mpirun);
2. определение ранга процесса (MPI_Comm_rank);
3. глобальные операции над группами процессов (MPI_Bcast, MPI_Reduce, MPI_Allgather);
4. точечные обмены (MPI_Send, MPI_Recv);
5. коллективные синхронизации.

В данной работе используются:

- **MPI_Bcast** — рассылка начального приближения вектора x ;
- **MPI_Allgather** — сбор новых значений вектора Якоби после каждой итерации;
- **MPI_Allreduce** — вычисление среднего отклонения сходимости.

2 МЕТОД ЯКОБИ РЕШЕНИЯ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ

2.1 Принципы математической реализации метода

Рассмотрим систему линейных алгебраических уравнений вида

$$Ax = b,$$

где $A \in R^{N \cdot N}$ — квадратная матрица порядка N , $b \in R^N$ — вектор правой части, $x \in R^N$ — искомый вектор решения.

Метод Якоби относится к классу простейших итерационных методов и основан на поэлементном обновлении решения. Пусть элементы матрицы A обозначены как a_{ij} . Тогда итерационная формула имеет вид:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, N.$$

Особенностью метода является то, что вычисление каждого компонента $x_i^{(k+1)}$ не зависит от значений $x_j^{(k+1)}$ при $j \neq i$. Это обеспечивает высокую степень независимости вычислений, поскольку вычисление строк матрицы может выполняться независимо.

2.2 Алгоритмическая сложность

В последовательной реализации вычислительная сложность одной итерации составляет $O(N)^2$, что обусловлено необходимостью формировать сумму по всем элементам строки, за исключением диагонального. При параллельной реализации, использующей p процессов и равномерное распределение строк матрицы, вычислительная сложность одной итерации принимает вид: $O\left(\frac{N^2}{p}\right) + O(N \cdot \log_2 p)$, где второй член связан с издержками коммуникаций, возникающими при выполнении коллективной операции

обмена вектором решения (например, MPI_Allgather).

2.3 Условия сходимости

Метод Якоби обладает гарантированной сходимостью при выполнении одного из следующих условий:

- матрица A является строго диагонально доминирующей: $|a_{ii}| > \sum_{i \neq j} |a_{ij}|, \forall i$;
- матрица является симметричной и положительно определённой;
- существует эквивалентное преобразование системы, приводящее её к одному из указанных видов.

В представленной реализации формирование диагонального преобладания обеспечивается программно посредством увеличения диагонального элемента.

3 ПАРАЛЛЕЛЬНАЯ РЕАЛИЗАЦИЯ МЕТОДА ЯКОБИ

3.1 *Декомпозиция по данным*

Реализация основана на равномерном распределении строк матрицы A и соответствующих элементов вектора правой части b между процессами. Каждому процессу передается блок из N/p строк, на основании которого он вычисляет свой участок вектора решения.

3.2 *Разделение матрицы между процессами*

В рассматриваемой программе каждый процесс выделяет область памяти под локальный фрагмент матрицы:

```
int local_n = N / size;  
double *A = malloc(local_n * N * sizeof(double));
```

Таким образом, каждый процесс хранит полный набор столбцов, но только часть строк. Данный подход упрощает реализацию вычислений и позволяет использовать прямую индексацию элементов.

3.3 *Межпроцессорное взаимодействие*

На каждой итерации выполняются три основных шага:

Вычисление локальных значений вектора решения:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1, j \neq z}^N a_{ij} x_j^{(k)}}{a_{i,z}},$$

где z — это индекс столбца матрицы A , соответствующий элементу $a_{i,z}$, используемому для нормировки при вычислении компоненты $x_i^{(k+1)}$ на i -й итерации.

Сбор обновлённого локального решения всеми процессами: используется коллективная операция `MPI_Allgather` для формирования полного вектора $x^{(k+1)}$.

Оценка сходимости:

с помощью `MPI_Allreduce` вычисляется норма разности $|x^{(k+1)} - x^k|$.

4 РАЗРАБОТКА MPI-ПРОГРАММЫ

Разработка параллельной программы для реализации метода Якоби включала последовательность этапов, соответствующих типовой структуре MPI-приложения. На первом этапе выполнялась инициализация среды MPI и определение количества процессов, участвующих в вычислениях. Далее осуществлялось формирование исходных данных: матрицы A коэффициентов и вектора правой части b , а также выбор начального приближения для вектора решения. Генерация структуры данных представлена в *Приложении [1]*.

После подготовки входных данных реализовывался итерационный процесс метода Якоби. Каждому процессу передавалась определённая часть строк матрицы, что обеспечивало равномерное распределение вычислительной нагрузки. В ходе итераций каждый процесс вычислял локальные значения вектора решения на основе доступных строк матрицы и текущего глобального вектора, после чего выполнялся коллективный обмен данными между всеми процессами. Используемые коллективные коммуникации (в частности, рассылка начального приближения и сбор обновлённого вектора решений) подробно показаны в *Приложении [2]*.

Контроль сходимости осуществлялся путём глобального вычисления нормы изменения вектора решения. Если изменение становилось меньше заданного порога, итерационный процесс прекращался. После завершения расчётов результаты собирались на одном процессе и выводились пользователю. Завершающим этапом являлось освобождение выделенной памяти и корректное завершение работы MPI-среды.

5 ЭКСПЕРИМЕНТАЛЬНЫЕ ИССЛЕДОВАНИЯ

5.1 Вычислительная платформа (кластер Pine)

Экспериментальная часть работы была выполнена на вычислительном кластере Pine (pine.crpt.sibsutis.ru). Кластер функционирует под управлением операционной системы GNU/Linux и системы управления заданиями SLURM, обеспечивающей планирование, мониторинг и распределение вычислительных ресурсов. Основу кластера составляют серверы с процессорами Intel Xeon Gold 5218N и Intel Xeon Gold 6230N, содержащие от 16 до 20 физических ядер (до 64–80 MPI-процессов на узел) и оборудованные высокообъемной оперативной памятью (до 390 GB). Доступ к системе осуществляется по SSH через головной узел, который работает в круглосуточном режиме. Постановка задач на выполнение производится через SLURM посредством job-скриптов, позволяющих задавать количество узлов, число MPI-процессов и параметры среды выполнения.

5.2 Методика проведения экспериментов

Эксперименты проводились для различных размеров системы $N = 28032$, 70048 и числа процессов $p = 8, 16, 32$. Целью исследования являлось определение влияния числа процессов (p) на время выполнения итерационного метода Якоби. Для фиксации времени использовалась внутренняя функция измерения времени MPI, позволяющая получить высокоточное значение длительности выполнения итерационного цикла. Для каждого набора параметров измерялось полное время решения системы, обозначаемое как $T(p, N)$. Полученные результаты представлены в таблице 5.1.

Число процессов p	Время, сек ($N = 28032$)	Время, сек ($N = 70048$)
8	30.3083	108.2693
16	12.8708	40.2546

32	5.0372	14.7736
----	--------	---------

Таблица 5.1 – Время выполнения программы Якоби MPI

5.3 Расчёт ускорения

Для оценки эффективности параллельного выполнения вычислялось ускорение, показывающее, насколько быстрее программа работает при использовании нескольких процессов. Значение ускорения определялось по стандартной формуле:

$$S_{(p)} = \frac{T(1)}{T(p)},$$

где:

- $S_{(p)}$ — ускорение программы при использовании p процессов.
- $T_{(1)}$ — время выполнения задачи на одном процессе.
- $T_{(p)}$ — время выполнения задачи при использовании p процессов.

Рост ускорения при увеличении числа процессов отражает способность метода и реализации эффективно использовать ресурсы параллельной системы. Более высокие значения ускорения наблюдались для больших систем, где вычислительные операции доминировали над коммуникационными издержками.

Результирующие значения приведены в таблице 5.2.

Число процессов p	$S(p)$ для $N=28032$	$S(p)$ для $N=70048$
8	1.000	1.000
16	2.354	2.689
32	6.023	7.328

Таблица 5.2 – Ускорение метода Якоби

5.4 *Влияние размера задачи на масштабируемость*

Анализ экспериментальных данных показывает, что увеличение размерности задачи существенно повышает уровень декомпозиции задачи. Для большего N наблюдается более выраженный рост ускорения. Это объясняется снижением относительного влияния коммуникационных операций, доля которых уменьшается по отношению к числу арифметических операций *Приложение [3]*.

Для размера $N = 70048$ ускорения выше, чем для $N = 28032$, что подтверждает теоретическое утверждение: итерационные методы, такие как метод Якоби, демонстрируют лучшую масштабируемость при увеличении размерности системы.

ВЫВОД

В ходе выполнения курсовой работы была разработана и исследована параллельная программа для решения систем линейных алгебраических уравнений методом Якоби с использованием технологии MPI. В работе последовательно рассмотрены архитектуры параллельных вычислений, особенности организации вычислений в модели распределённой памяти, а также принципы построения гибридных и многопроцессорных систем. На основе теоретического анализа была обоснована целесообразность применения метода Якоби для параллельной реализации ввиду его хорошей декомпозиции и минимальной зависимости между вычислениями на каждой итерации.

В рамках практической части была выполнена разработка MPI-программы, включающая подготовку исходных данных, распределение матрицы и вектора правой части между процессами, реализацию итерационного цикла и контроль сходимости. Реализация учитывает особенности коммуникаций и обеспечивает корректный обмен промежуточными результатами между всеми процессами.

Проведённые экспериментальные исследования позволили оценить ускорение, масштабируемость и эффективность разработанного алгоритма. Полученные результаты показали, что MPI-реализация демонстрирует близкое к линейному ускорение при увеличении числа процессов и особенно эффективно работает на задачах большой размерности. Это подтверждает корректность программной реализации и применимость разработанного решения для вычислений на кластерных системах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гергель, В. П. Теория и практика параллельных вычислений / В. П. Гергель. — Москва: МГУ, 2020. — 512 с.
2. Гергель, В. П., Стронгин, Р. Г. Основы параллельных вычислений для многопроцессорных вычислительных систем / В. П. Гергель, Р. Г. Стронгин. — Москва: Наука, 2018. — 384 с.
3. Эндрос, Г. Р. Основы многопоточного, параллельного и распределённого программирования. — М.: Наука, 2003.
4. Карниадакис, Г., Кирби, Р. Параллельные научные вычисления на C++ и MPI: Бесшовный подход к параллельным алгоритмам и их реализации / Г. Карниадакис, Р. Кирби. — Кембридж: Cambridge University Press, 2003. — 616 с. — ISBN 0-521-52039-0.
5. Документация кластера Pine ИВЦ СибГУТИ [Электронный ресурс]. – 2024. – URL: <http://pine.cpct.sibsutis.ru> (дата обращения: 10.9.2025).
6. Разработка параллельных алгоритмов методов решения систем линейных алгебраических уравнений [Электронный ресурс]. — URL: <https://cyberleninka.ru/article/n/razrabotka-parallelnyh-algoritmov-metodov-resheniya-sistem-lineynyh-algebraicheskikh-uravneniy/viewer> (дата обращения: 02.11.2025).

6 ПРИЛОЖЕНИЯ

1) Код генерации входных данных (матрицы A и вектора b)

```
// jacobi.c
srand(time(NULL) + rank);

for (int i = 0; i < local_n; i++)
{
    int global_i = rank * local_n + i;
    double sum = 0.0;
    for (int j = 0; j < N; j++)
    {
        if (j != global_i)
        {
            A[i * N + j] = (rand() % 2000 - 1000) / 100.0;
            sum += fabs(A[i * N + j]);
        }
    }

    A[i * N + global_i] = sum * (1.001 + (rand() % 10) / 1000.0);
    b[i] = (double)(rand() % 1000) / 10.0;
}
```

2) Вычислительное ядро метода Якоби и организация коммуникаций MPI

```
for (iter = 0; iter < max_iter; iter++)
{
    for (int i = 0; i < local_n; i++)
    {
        int global_i = rank * local_n + i;
        double sum = 0.0;
```

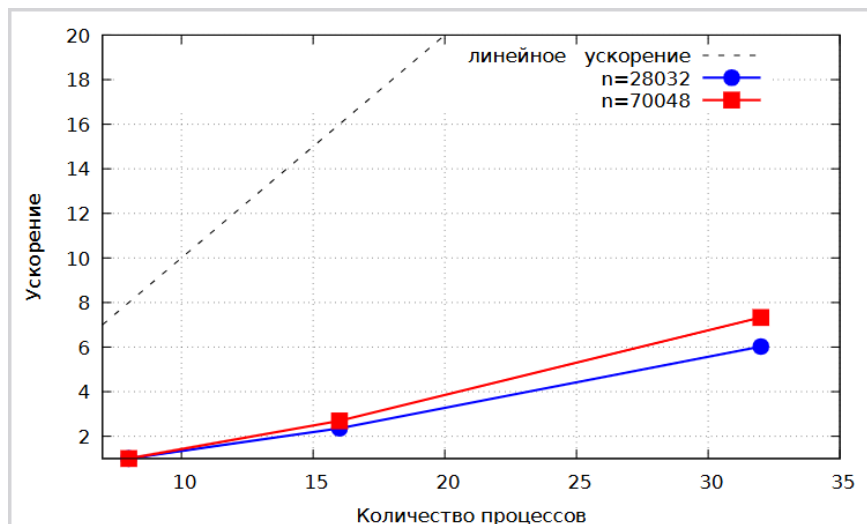


```

for (int j = 0; j < N; j++){
    if (j != global_i)
    {
        sum += A[i * N + j] * x[j];
    }
}
local_x_new[i] = (b[i] - sum) / A[i * N + global_i];
}
MPI_Allgather(local_x_new, local_n, MPI_DOUBLE,
x_new, local_n, MPI_DOUBLE, MPI_COMM_WORLD);
double local_diff = 0.0;
for (int i = rank * local_n; i < (rank + 1) * local_n; i++)
{
    local_diff += fabs(x_new[i] - x[i]);
}
double global_diff;
MPI_Allreduce(&local_diff, &global_diff, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
global_diff /= N;
for (int i = 0; i < N; i++)
{
    x[i] = x_new[i];
}
}

```

3) Результаты экспериментов:



4) Исходный код:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int N = 50000;
    if (argc > 1)
        N = atoi(argv[1]);

    if (N <= 0)
    {
        if (rank == 0)
            printf("Error: size must be > 0\n");
        MPI_Finalize();
        return 1;
    }

    if (N % size != 0)
    {
        if (rank == 0)
            printf("Error: size must be divisible by %d\n", size);
        MPI_Finalize();
    }
}
```

```

        return 1;
    }

    int local_n = N / size;
    int max_iter = 100;

    double *A = malloc(local_n * N * sizeof(double));
    double *b = malloc(local_n * sizeof(double));
    double *x = malloc(N * sizeof(double));
    double *x_new = malloc(N * sizeof(double));
    double *local_x_new = malloc(local_n * sizeof(double));

    if (A == NULL || b == NULL || x == NULL || x_new == NULL || local_x_new ==
        NULL)
    {
        printf("Process %d: out of memory\n", rank);
        MPI_Finalize();
        return 1;
    }

    srand(time(NULL) + rank);

    for (int i = 0; i < local_n; i++)
    {
        int global_i = rank * local_n + i;
        double sum = 0.0;

        for (int j = 0; j < N; j++)
        {
            if (j != global_i)
            {
                A[i * N + j] = (rand() % 2000 - 1000) / 100.0;
                sum += fabs(A[i * N + j]);
            }
        }

        A[i * N + global_i] = sum * (1.001 + (rand() % 10) / 1000.0);
        b[i] = (double)(rand() % 1000) / 10.0;
    }

    if (rank == 0)
    {
        for (int i = 0; i < N; i++)
        {
            x[i] = (double)(rand() % 10000) / 100.0;
        }
    }

    MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

```

```

double start_time = 0;
if (rank == 0)
{
    start_time = MPI_Wtime();
}

int iter;

for (iter = 0; iter < max_iter; iter++)
{
    for (int i = 0; i < local_n; i++)
    {
        int global_i = rank * local_n + i;
        double sum = 0.0;

        for (int j = 0; j < N; j++)
        {
            if (j != global_i)
            {
                sum += A[i * N + j] * x[j];
            }
        }

        local_x_new[i] = (b[i] - sum) / A[i * N + global_i];
    }

    MPI_Allgather(local_x_new, local_n, MPI_DOUBLE,
                  x_new, local_n, MPI_DOUBLE, MPI_COMM_WORLD);

    double local_diff = 0.0;
    for (int i = rank * local_n; i < (rank + 1) * local_n; i++)
    {
        local_diff += fabs(x_new[i] - x[i]);
    }

    double global_diff;
    MPI_Allreduce(&local_diff, &global_diff, 1, MPI_DOUBLE, MPI_SUM,
                  MPI_COMM_WORLD);
    global_diff /= N;

    for (int i = 0; i < N; i++)
    {
        x[i] = x_new[i];
    }
}

double end_time = 0;
if (rank == 0)

```

```
{
    end_time = MPI_Wtime();
    double total_time = end_time - start_time;
    printf("JACOBI: n=%d, processes=%d, time=%.4f sec, iter=%d\n",
          N, size, total_time, max_iter);
}

free(A);
free(b);
free(x);
free(x_new);
free(local_x_new);

MPI_Finalize();
return 0;
}
```